Tiered-Autonomy Agentic Decision Support for Shipment Delay Exception Management: Architecture and Simulation-Based Evaluation

Sophia Gazula

Independent Researcher, Chicago, Illinois, USA

Correspondence: sophiegazula@gmail.com

Abstract

Shipment delays are among the most frequent and costly exceptions in manufacturing supply chains, yet many organizations still detect and resolve them through threshold-based alerts and manual planner triage. This paper presents the design and evaluation of a delay resolution agent: an agentic decision system that predicts international shipment delays, scores their operational severity, selects a response under a tiered-autonomy policy, and executes actions through integrations with a transportation management system (TMS) and a team messaging platform. The architecture deliberately combines a Random Forest delay classifier with a transparent rule-based severity layer and a decision policy engine, rather than relying on a large language model for operational decisions, in order to preserve auditability, latency, and cost predictability. Low-risk actions are executed automatically, medium-risk actions require planner approval, and high-impact disruptions are escalated for human-led resolution. The system is evaluated on a synthetic dataset of 48,000 international shipments modeled on an automotive parts inbound network, together with a discrete-event simulation of planner workflows. On a held-out six-month test period, the classifier identified 68.1% of delayed shipments in transit (precision 0.58, ROC-AUC 0.87), compared with 20.7% for a carrier-ETA rule. In the simulated base case, mean time to resolution (MTTR) for delayed shipments fell from 33.8 to 16.4 hours (−51%) and planner workload fell by 39%; across staffing and handling-time scenarios the MTTR reduction ranged from 31% to 93%. The paper contributes a replicable reference architecture, an open evaluation design, and a candid discussion of the conditions under which tiered autonomy pays off.

Keywords: agentic AI; supply chain exception management; shipment delay prediction; human-in-the-loop automation; random forest; discrete-event simulation; logistics

1. Introduction

Global supply chains have become more interconnected and, as a result, more exposed to disruption. Transportation delays, port congestion, weather events, customs holds, and carrier capacity shortages propagate through tightly coupled networks and can quickly affect production schedules and customer commitments (Ivanov & Dolgui, 2020). Artificial intelligence has been widely proposed as a way to strengthen supply chain risk management, but much of the published work concentrates on prediction and risk identification rather than on closing the loop to action (Baryannis et al., 2019).

In practice, delay exceptions are still commonly handled through a manual process: a planning system raises an alert when a shipment milestone is missed, a planner investigates the cause, weighs options such as expediting or reallocating inventory, and then updates systems and notifies stakeholders. This process scales poorly as shipment volumes grow, produces inconsistent decisions across planners, and tends to react only after a delay has become visible.

The term *agentic AI* describes systems that perceive their environment, make decisions in pursuit of goals, and act on that environment with some degree of autonomy (Wooldridge & Jennings, 1995; Russell & Norvig, 2020). Recent work on large language model (LLM) agents has renewed interest in this idea (Wang et al., 2024; Yao et al., 2023b). However, enterprise operations place strong demands on auditability, determinism, and cost control, and an agent does not need to be LLM-centric to be useful. A system that combines a well-validated predictive model with explicit decision rules and controlled execution can deliver many of the operational benefits of autonomy while remaining straightforward to govern.

Machine learning for delay and estimated-time-of-arrival (ETA) prediction is well studied (Balster et al., 2020; Ni et al., 2020), but published accounts rarely describe how such predictions are connected to automated actions, how autonomy is bounded, or how the resulting workflow changes resolution times and planner workload. This paper addresses that gap through the following research questions:

RQ1. How can delay prediction be combined with severity assessment and policy-based action to form a closed-loop exception management agent?

RQ2. What governance mechanisms allow such an agent to act autonomously on routine cases while keeping humans in control of consequential ones?

RQ3. How does such an agent affect time to resolution and planner workload, and under what operating conditions are the gains largest?

The paper makes four contributions: (1) a reference architecture for delay exception management organized into sense, predict, assess, decide, act, and learn stages; (2) a tiered-autonomy policy that applies established levels-of-automation concepts (Parasuraman et al., 2000) to logistics exceptions; (3) a reproducible evaluation combining a synthetic shipment dataset, a time-based model comparison, and a discrete-event simulation of planner workflows with sensitivity analysis; and (4) practical lessons and a roadmap for incorporating LLM-based reasoning where it adds value.

Section 2 reviews related work. Section 3 describes the system architecture. Section 4 presents the experimental setup, and Section 5 reports results. Section 6 discusses implications, Section 7 addresses limitations, Section 8 outlines future research, and Section 9 concludes.

2. Background and Related Work

2.1 Supply chain exception and risk management

Exception management has evolved from manual monitoring, through enterprise resource planning (ERP), transportation management (TMS), and warehouse management (WMS) systems that provide structured workflows and visibility, to rule-based alerting that triggers notifications when operational thresholds are breached. Rule-based approaches are transparent but brittle: thresholds are static, they ignore context such as inventory position or customer commitments, and they generate alerts rather than resolutions. Research on supply chain risk management has increasingly turned to AI techniques to anticipate and assess disruptions (Baryannis et al., 2019), and work on supply chain resilience and viability has emphasized the need for faster, adaptive responses to disruption (Ivanov & Dolgui, 2020).

2.2 Machine learning for delay and ETA prediction

Supervised learning has been applied extensively across supply chain management, including demand forecasting, supplier risk, and transportation (Ni et al., 2020). In transportation specifically, Balster et al. (2020) developed a machine-learning model to predict ETAs in intermodal networks, demonstrating that operational data can support accurate arrival-time estimates. Tree-based ensembles such as Random Forests

(Breiman, 2001) are well suited to these problems because they handle mixed numeric and categorical features, capture non-linear interactions, and are relatively robust to noisy inputs. Most of this literature, however, stops at prediction; the decision of what to do with a predicted delay is left to human planners.

2.3 Agents and multi-agent systems in supply chains

Software agents have a long history in supply chain research. Fox et al. (2000) proposed an agent-oriented architecture in which specialized agents coordinate supply chain functions such as order acquisition, logistics, and scheduling. The broader agent literature defines agents by their autonomy, reactivity, and goal-directed behavior (Wooldridge & Jennings, 1995). These foundations remain relevant: an exception management agent must sense events, reason about them, and act, regardless of which modeling techniques it uses internally.

2.4 Large language model–based agents

Large language models have enabled a new generation of agents. Chain-of-thought prompting improves multi-step reasoning (Wei et al., 2022); ReAct interleaves reasoning with tool use (Yao et al., 2023b); Tree of Thoughts explores multiple reasoning paths (Yao et al., 2023a); and retrieval-augmented generation grounds outputs in external knowledge (Lewis et al., 2020). Surveys document rapid growth in LLM-based autonomous agents (Wang et al., 2024). For operational decisions with direct financial consequences, however, LLM agents raise practical concerns, including non-deterministic outputs, difficulty auditing reasoning, latency, and per-decision cost. These concerns motivate the hybrid design described in this paper.

2.5 Human–automation interaction and AI governance

Parasuraman et al. (2000) proposed that automation can be applied at different levels across information acquisition, analysis, decision selection, and action implementation, and that the appropriate level depends on the consequences of error. Human-centered AI frameworks argue for designs that combine high automation with high human control (Shneiderman, 2020), and the NIST AI Risk Management Framework provides guidance for governing AI systems across their life cycle (National Institute of Standards and Technology, 2023). Experience with production machine learning also shows that the surrounding infrastructure for data, monitoring, and configuration often dominates long-term maintenance effort (Sculley et al., 2015). Table 1 summarizes how the approach in this paper relates to earlier forms of exception management.

Table 1. Comparison of exception management approaches

Dimension	Rule-based alerting	Predictive analytics	Closed-loop agent (this work)
Trigger	Static threshold breach	Model-predicted risk	Model-predicted risk + severity
Decision	Human planner	Human planner, informed by prediction	Policy engine; human for higher tiers
Action execution	Manual	Manual	Automated via system integrations (Tier 1)
Human role	Performs all steps	Interprets predictions	Approves, escalates, overrides
Adaptation	Manual rule edits	Periodic retraining	Retraining + override feedback
Auditability	High	Moderate	High (logged rules and actions)

3. System Architecture

The delay resolution agent is organized as a pipeline of six stages (Figure 1). Each stage has a narrow responsibility and a well-defined interface, which simplifies testing, auditing, and incremental improvement.

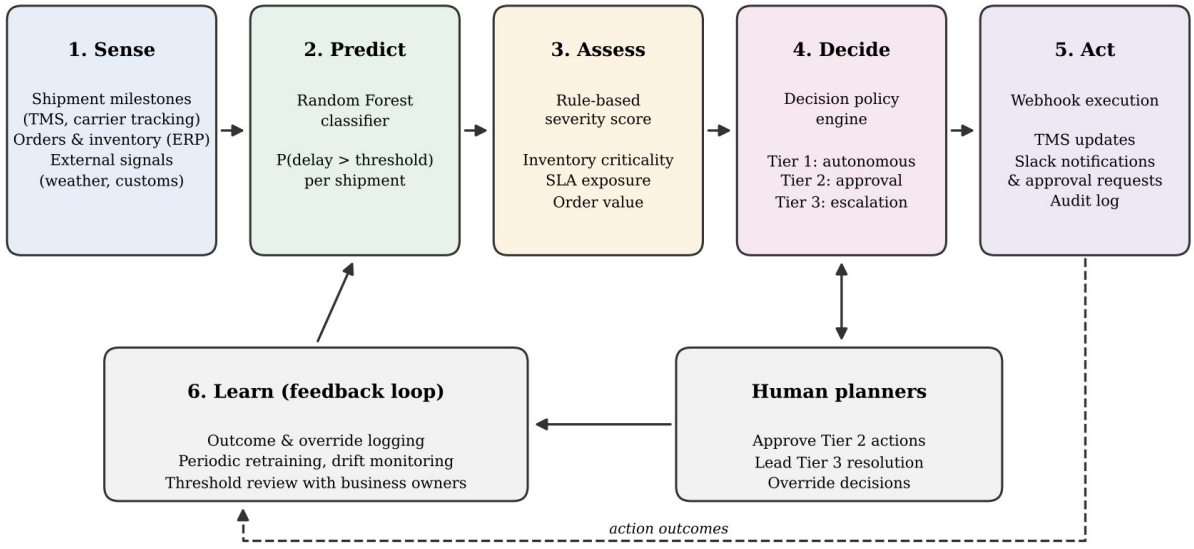


Figure 1. Architecture of the delay resolution agent

3.1 Sense: data ingestion

The sensing stage collects shipment milestone events from the TMS and carrier tracking feeds, order and inventory context from the ERP system, and external signals such as weather and customs status. Records are refreshed every four hours and joined at the shipment level to form the input to the prediction stage.

3.2 Predict: delay classification

A Random Forest classifier (Breiman, 2001) estimates, for each in-transit shipment, the probability *p* that it will arrive more than 48 hours later than its planned arrival date. A Random Forest was selected because the input data are tabular with mixed feature types, because the model captures interactions (for example, between carrier and lane) without manual feature engineering, and because feature importances provide a starting point for explaining predictions to planners. Per-prediction explanations can additionally be produced with SHAP values (Lundberg & Lee, 2017).

3.3 Assess: rule-based severity scoring

A predicted delay is not necessarily an important one. The assessment stage converts the delay probability into a severity score that reflects business impact:

$$S = p \times (0.5 \cdot C_{inv} + 0.3 \cdot C_{sla} + 0.2 \cdot C_{val})$$

Inventory criticality, C_{inv} = min(E[delay days] / days of supply at destination, 1), measures whether on-hand inventory can absorb the expected delay, where the expected delay is the delay probability multiplied by the median delay observed on that lane in the training data. SLA exposure, C_{sla}, equals 1 for shipments linked to a customer service-level agreement and 0.2 otherwise. Value exposure, C_{val}, is the order value divided by the 95th percentile of order values, capped at 1. The weights are business policy rather than learned parameters, and keeping this layer rule-based allows planners and managers to read, question, and adjust exactly how impact is judged.

3.4 Decide: tiered-autonomy policy engine

The decision policy engine maps each flagged shipment to one of three autonomy tiers based on its severity score and the cost and reversibility of the candidate action (Table 2). This operationalizes the principle that the appropriate level of automation should depend on the consequences of error (Parasuraman et al., 2000).

Table 2. Tiered-autonomy decision policy

Tier	Criteria	Typical actions	Human role
Tier 1 — Autonomous	$S < 0.20$; action limited to record updates and notifications (no spend)	Update ETA in TMS; notify stakeholders; log exception	Informed after the fact; can override
Tier 2 — Approval	$0.20 \le S < 0.40$	Propose expedite, reroute, or inventory reallocation; request approval via messaging	Approves or rejects proposed action
Tier 3 — Escalation	$S \ge 0.40$	Open escalation with context package (prediction, drivers, options)	Leads resolution

3.5 Act: execution through integrations

Actions are executed through webhooks to the TMS and to Slack. Tier 1 actions update records and post notifications directly; Tier 2 actions post a structured approval request that planners can accept or reject; Tier 3 actions open an escalation containing the prediction, its main drivers, and candidate responses. Every action, approval, and override is written to an audit log with a timestamp and the inputs that produced it.

3.6 Learn: feedback and monitoring

Realized arrival dates and planner overrides are logged and used to evaluate prediction quality over time. The model is retrained monthly, input distributions are monitored for drift, and tier thresholds are reviewed

periodically with business owners. Planner overrides are treated as a signal that either the model or the severity rules require adjustment.

4. Experimental Setup

4.1 Synthetic shipment dataset

Because operational shipment data are commercially sensitive, the system is evaluated on a synthetic dataset designed to reproduce the structure of an automotive parts inbound network. The dataset contains 48,000 international shipments dispatched between January 2024 and December 2025 across 32 lanes (eight origin countries in Asia and Europe to four North American distribution centers) and 12 carriers (eight ocean, four air); 72% of shipments move by ocean. Delay is generated from a latent process driven by carrier reliability, lane variability, weather severity (with a typhoon-season effect for Asian origins), customs holds, carrier-level congestion shocks that follow a first-order autoregressive process, holiday windows (Lunar New Year, Golden Week, and year-end), and peak season, plus random noise. A shipment is labeled *delayed* if it arrives more than 48 hours after its planned arrival; 15.1% of shipments meet this definition. Each shipment also carries business context (days of inventory supply at destination, an SLA-linkage flag, and order value) used only by the severity layer. Predictions are made at a checkpoint 40% of the way through planned transit, using only information available at that time. Table 3 lists the features.

Table 3. Features available to the delay classifier at the prediction checkpoint

Feature group	Features
Shipment	Mode, origin, destination, planned transit days, weight
Carrier	Carrier identity, historical on-time rate, share of late arrivals in the last 30 days
Lane	Historical transit-time variability of the origin–destination pair
Temporal	Week of year, month, day of week, holiday-window indicator
External	Weather severity index on route, customs hold indicator
Milestone progress	Hours by which intermediate milestones are running late

4.2 Model training and validation

To avoid temporal leakage, the data were split chronologically into a training window (January 2024–March 2025; 29,940 shipments), a validation window (April–June 2025; 6,004 shipments), and a held-out test window (July–December 2025; 12,056 shipments, of which 2,633 were delayed). Random Forest hyperparameters were selected by three-fold time-series cross-validation on the training window, optimizing precision–recall AUC; the selected model uses 300 trees, unrestricted depth, a minimum of five samples per leaf, and balanced class weights. The decision threshold for each model was chosen to maximize F1 on the validation window (0.56 for the Random Forest), after which models were refit on training and validation data combined. The Random Forest was compared with two baselines: a carrier-ETA rule that flags a shipment when the carrier-reported arrival estimate slips by more than 48 hours, and a class-weighted logistic regression on the same features. Results are reported as precision, recall, F1-score, ROC-AUC, and precision–recall AUC (PR-AUC).

4.3 Workflow simulation

To estimate operational impact, the test window was replayed through a discrete-event simulation of two workflows. In the **manual workflow**, an exception is created when the carrier-ETA rule fires at the prediction checkpoint or, for delays the rule misses, when the planned arrival date passes; every exception is

handled by a planner. In the **agent workflow**, the agent runs every four hours; flagged shipments are routed by tier, and delays the model misses are caught at the planned arrival date and handled manually, as in the baseline. Planners work Monday to Friday, 08:00–18:00, and take exceptions first-in, first-out. Handling times are drawn from lognormal distributions (coefficient of variation 0.5) with means of 3.0 hours for a full investigation, 1.0 hour to dismiss a false alarm, 0.5 hours to approve a Tier 2 proposal, and 0.3 hours to reject one; Tier 1 actions complete automatically in 5–20 minutes. The base case uses eight planners, which places the manual workflow at roughly 75% utilization. MTTR is the elapsed time from exception creation to resolution. Each scenario was replicated ten times, and a sensitivity analysis varied staffing (7, 8, and 10 planners) and handling times (70%, 100%, and 130% of base values). These parameters are assumptions, chosen to be plausible for a mid-sized inbound logistics team.

5. Results

5.1 Predictive performance

Table 4 reports performance on the held-out test window. The carrier-ETA rule is highly precise (0.93) but identifies only 20.7% of delayed shipments while they are still in transit. Both learned models identify far more delays: the Random Forest achieves recall of 0.68 with precision of 0.58, and the logistic regression trades lower precision (0.51) for higher recall (0.76). The two learned models are effectively tied on ranking quality (ROC-AUC 0.87, PR-AUC 0.70), with the Random Forest slightly ahead on F1 (0.63 vs. 0.61). The Random Forest was retained for deployment because its higher precision reduces false alarms reaching planners. The most important features were milestone lateness, the carrier's recent late-arrival share, and the carrier's historical on-time rate.

Table 4. Delay classification performance on held-out test data (July–December 2025)

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Carrier-ETA rule (baseline)	0.930	0.207	0.338	—	—
Logistic regression	0.513	0.762	0.613	0.871	0.704
Random Forest (selected)	0.582	0.681	0.628	0.869	0.700

5.2 Operational impact

Of the shipments flagged by the agent, 33.3% were assigned to Tier 1, 43.5% to Tier 2, and 23.1% to Tier 3. Because the model flags more shipments than the rule, the agent workflow generated more exceptions (3,918 vs. 2,674), but 26.2% of them were resolved automatically and many others required only a short approval. Table 5 summarizes the base-case results. Mean time to resolution for delayed shipments fell from 33.8 to 16.4 hours (−51%), and median time to resolution fell from 30.1 to 9.7 hours (−68%). Total planner handling time fell by 39%, from 7,922 to 4,799 hours over the six-month test window. Variation across the ten replications was small (standard deviation of MTTR 1.8 hours for the manual workflow and 0.2 hours for the agent workflow).

Table 5. Simulated operational outcomes, base case (8 planners; mean of 10 replications)

Measure	Manual workflow	Agent workflow	Change
Delays identified while in transit	20.7%	68.1%	+47.4 pts
Exceptions created	2,674	3,918	+46.5%
Exceptions resolved automatically	0%	26.2%	—
Mean time to resolution, delayed shipments (h)	33.8	16.4	−51.4%
Median time to resolution, delayed shipments (h)	30.1	9.7	−68.0%
Planner handling hours	7,922	4,799	−39.4%

5.3 Sensitivity analysis

Table 6 shows that the size of the benefit depends strongly on how heavily loaded the planning team is. When the team has ample spare capacity (10 planners, short handling times), the agent reduces MTTR by 31%, mainly by automating Tier 1 cases and shortening Tier 2 handling. As load increases, queues in the manual workflow lengthen sharply, and the agent's reduction in planner workload translates into much larger MTTR gains: 73% with seven planners at base handling times, and up to 93% when handling times are also 30% longer. In every scenario the agent workflow resolved delays faster than the manual workflow.

Table 6. Sensitivity of mean time to resolution (hours) to staffing and handling time

Planners	Handling time	Manual MTTR	Agent MTTR	Reduction
7	70%	23.1	14.6	37%
7	100%	64.5	17.4	73%
7	130%	348.8	25.9	93%
8 (base)	70%	21.5	14.2	34%
8 (base)	100%	33.8	16.4	51%
8 (base)	130%	158.0	20.2	87%
10	70%	19.9	13.8	31%
10	100%	24.6	15.4	38%
10	130%	41.0	17.5	57%

6. Discussion

6.1 Answers to the research questions

Regarding RQ1, the results show that a closed-loop agent can be assembled from conventional components: a validated classifier supplies the probability of delay, a rule layer translates that probability into business impact, and a policy engine selects and executes a response. Regarding RQ2, tiered autonomy, a restriction of autonomous actions to no-spend record updates and notifications, approval workflows in a familiar messaging tool, and a complete audit log allow routine cases to be automated while consequential decisions remain with planners. Regarding RQ3, the simulation indicates that the agent roughly halves mean time to resolution and reduces planner workload by about two-fifths in the base case, and that the gains are largest when the planning team is close to capacity, which is precisely when disruptions cluster.

6.2 Where the benefit comes from

The improvement has two sources. First, the classifier catches more than three times as many delays while shipments are still in transit, so more exceptions start early rather than when a planned arrival is missed. Second, tiering removes planner effort: Tier 1 cases need no planner time and Tier 2 cases need only a short approval. The second effect explains why the benefit grows with load. The cost of the agent is a higher number of exceptions, many of them false alarms; the tier structure is what keeps this cost manageable, since low-severity false alarms are absorbed automatically.

6.3 Why not an LLM-first design?

The core decision in this system—whether a shipment will be late and how much that matters—is a structured prediction over tabular data, a task for which tree ensembles and linear models are well established. The near-identical ROC-AUC of the Random Forest and logistic regression also suggests that model choice matters less than the surrounding decision design. An LLM-first design would add non-determinism, make decisions harder to audit, and introduce latency and per-decision cost without a clear accuracy benefit for this task. LLMs are better suited to complementary roles: interpreting unstructured inputs such as carrier emails and customs notices, drafting stakeholder communications, and explaining decisions in natural language.

6.4 Governance lessons

- **Start with low autonomy and expand it.** Launching with conservative Tier 1 limits and widening them as override rates fall builds planner trust.
- **Let the business own thresholds.** Severity weights and tier boundaries are business policy, not model parameters, and should be maintained by operations owners.
- **Log everything.** A full record of inputs, decisions, approvals, and overrides supports auditing, debugging, and retraining.
- **Keep a manual override.** Planners must be able to pause automation or override any decision at any time.

7. Limitations and Threats to Validity

- **Synthetic data.** The dataset is generated from an assumed delay process. Real networks may contain patterns, data-quality problems, and shifts that are not represented, and real-world model performance may be lower.
- **Simulation assumptions.** Handling times, staffing, working hours, and first-in, first-out queuing are assumptions. The sensitivity analysis shows that results depend strongly on team load, so the absolute MTTR figures should be read as illustrative.
- **Simplified actions.** The simulation measures time to resolution, not the quality or cost of the chosen mitigation, and assumes that approved actions succeed.
- **Label definition.** Results depend on the 48-hour delay threshold and on the choice of a prediction checkpoint at 40% of transit.
- **No field validation.** The architecture has not yet been evaluated against live operational data; a field study is the most important next step.

8. Future Research Directions

- **Field validation.** Evaluate the agent on live shipment data, ideally with a staggered rollout across lanes or sites to obtain causal estimates of impact.
- **LLM-assisted reasoning.** Use language models with retrieval-augmented generation (Lewis et al., 2020) over historical incidents to interpret unstructured signals and draft resolution options, while retaining the rule-based policy for final decisions.
- **Learning action policies.** Apply contextual bandits or reinforcement learning to learn which mitigation actions work best for which exception types, using logged outcomes.
- **Multi-agent coordination.** Extend the design to coordinate with procurement, inventory, and customer service agents, building on agent-oriented supply chain architectures (Fox et al., 2000).

9. Conclusion

This paper presented a delay resolution agent that combines Random Forest delay prediction, rule-based severity scoring, and a tiered-autonomy policy engine to detect and resolve shipment delay exceptions, with execution through TMS and messaging integrations. On a synthetic six-month test period, the agent identified more than three times as many delays in transit as a carrier-ETA rule and, in simulation, reduced mean time to resolution by 51% and planner workload by 39% in the base case, with larger gains when planning teams are heavily loaded. The broader lesson is that agentic AI in enterprise operations can begin with transparent, auditable components and explicit limits on autonomy, and can then grow in capability as trust and evidence accumulate.

Declarations

Data and code availability. All data used in this study are synthetic. The data generator, model training code, and simulation code are available from the author on request.

Funding. This research received no external funding.

Competing interests. The author declares no competing interests.

Use of generative AI. Generative AI tools were used to assist with drafting and editing this manuscript and with preparing analysis code. The author reviewed and edited all content and takes full responsibility for it.

References

Balster, A., Hansen, O., Friedrich, H., & Ludwig, A. (2020). An ETA prediction model for intermodal transport networks based on machine learning. *Business & Information Systems Engineering, 62*(5), 403–416. https://doi.org/10.1007/s12599-020-00653-0

Baryannis, G., Validi, S., Dani, S., & Antoniou, G. (2019). Supply chain risk management and artificial intelligence: State of the art and future research directions. *International Journal of Production Research, 57*(7), 2179–2202. https://doi.org/10.1080/00207543.2018.1530476

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Fox, M. S., Barbuceanu, M., & Teigen, R. (2000). Agent-oriented supply-chain management. *International Journal of Flexible Manufacturing Systems, 12*(2–3), 165–188.

Ivanov, D., & Dolgui, A. (2020). Viability of intertwined supply networks: Extending the supply chain resilience angles towards survivability. A position paper motivated by COVID-19 outbreak. *International Journal of Production Research, 58*(10), 2904–2915. https://doi.org/10.1080/00207543.2020.1750727

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765–4774.

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

Ni, D., Xiao, Z., & Lim, M. K. (2020). A systematic review of the research trends of machine learning in supply chain management. *International Journal of Machine Learning and Cybernetics, 11*(7), 1463–1482. https://doi.org/10.1007/s13042-019-01050-0

Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans, 30*(3), 286–297. https://doi.org/10.1109/3468.844354

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems, 28*, 2503–2511.

Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction, 36*(6), 495–504. https://doi.org/10.1080/10447318.2020.1741118

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), 186345. https://doi.org/10.1007/s11704-024-40231-1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824–24837.

Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review, 10*(2), 115–152. https://doi.org/10.1017/S0269888900008122

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023a). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems, 36*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023b). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*.